

Experiment - 07

Ques Cheapest flight from source to destination with atmost k stops.

Algorithm:-

- ① We use Breadth first search in finding the minimum path as BFS always ~~guarant~~ guarantees the minimum possible path.
- ② create the graph using $\text{vector} < \text{vector} < \text{pair} < \text{int}, \text{int} > > >$
- ③ create a queue and push the source with 0 weight and 0 cost to itself.
- ④ keep a track of min cost
- ⑤ while (!q.empty())
we check every neighbour of curr node in: q
if (cost + price < minCost[nxt])
{
 minCost[nxt] = cost + price
 // and update in queue.
}
}
- ⑥ return minCost[dst].

```

#include <bits/stdc++.h> using namespace std;
int findKCheapest (int n, vector<vector<int>>& flights,
int src, int dst, int k) {
    vector<vector<pair<int,int>>> graph(n);
    for (auto & flight : flights) {
        graph[flight[0]].push_back({flight[1], flight[2]});
    }
    queue<vector<int>> q; // city, cost, stops
    q.push({src, 0, 0});
    vector<int> minCost(n, INT_MAX);
    minCost[src] = 0;
    while (!q.empty()) {
        auto curr = q.front();
        q.pop();
        int city = curr[0], cost = curr[1], stops = curr[2];
        if (stops > k) continue;
        for (auto & neighbour : graph[city]) {
            int nxt = neighbour.first;
            int price = neighbour.second;
            if (cost + price < minCost[nxt]) {
                minCost[nxt] = cost + price;
                q.push({nxt, cost + price, stops + 1});
            }
        }
    }
    return minCost[dst] == INT_MAX ? -1 : minCost[dst];
}

```

T.C :- $O(E \cdot K)$

S.C :- $O(E + V)$ - queue.